

Non-throwing container operations

Abstract

This paper explores alternatives for adding non-throwing container operations, namely alternatives to throwing exceptions from failing modifications. Based on LEWG feedback from Jacksonville 2018 meeting, the focus is on minor additions to existing container APIs, instead of completely-custom allocators or completely-new containers. This paper suggests an evolutionary step and asks LEWG to clarify that the step is in the right direction.

In a nutshell, despite not being 100% reflective on the LEWG guidance polls, we need to consider a bunch of things, and decide which ones we want:

1. Allow an allocator to fail without an exception, by adding a separate function for non-throwing allocation.
 - Make `allocator_traits` wrap allocators that don't opt in to that functionality, so that callers who absolutely don't want allocation exceptions can avoid them.
2. Add a non-throwing reserve operation to vector and string.
3. Add a non-failing `push_back` to vector and string. This function will be UB to call if capacity isn't sufficient. The non-throwing reserve can be used to ensure sufficient capacity without exceptions.
4.
 - a) Add a `push_back` that can fail to vector and string. This function will report an allocation error, but will do so without an exception.
 - b) Add a `push_back` that can fail to vector and string. This function will have no effects if allocation fails, and will not throw.
5. Decide what to do with non-contiguous containers, like lists and maps.

Introduction

The guidance polls in Jacksonville 2018 were as follows:

Simple status return bool `push_back(nothrow_t, const T&)`

SF	F	N	A	SA
0	1	3	6	9

Use a different function name bool `push_back_nothrow(const T&)`

SF	F	N	A	SA
1	10	8	3	1

A much less intrusive alternative: allow allocator to return `nullptr`, etc.

SF	F	N	A	SA
0	1	6	8	5

A minimal interface: add `try_reserve/reserve_nothrow/...`

SF	F	N	A	SA
1	7	12	1	0

A custom allocator with a failure callback. We don't know enough about this to provide an opinion.

Adding specialized containers

SF	F	N	A	SA
1	9	5	4	3

Language mode for "allocation doesn't throw"

SF	F	N	A	SA
4	4	2	5	7

Make throwing unspecified when allocation fails.

SF	F	N	A	SA
1	1	2	3	14

So what are you proposing?

Let's go through the enumerated items mentioned in the abstract one by one. It should be mentioned straight off the bat that I

haven't tried to specify noexcept-specifications in the suggestions; that requires a more careful look.

Allow an allocator to fail without an exception, by adding a separate function for non-throwing allocation.

For users who want to avoid exceptions (for whatever reasons), using types that don't throw is almost enough. It's not quite enough because containers do not invoke non-throwing operations on allocators (mostly because allocators don't have such operations). Without allocators, we do provide this functionality via `new(nothrow)`. I'm merely suggesting that we expose the same capability via an allocator interface, and amend the standard allocator to provide a native implementation of a non-throwing allocation.

To remind readers of the original rationale, this is desirable because we are talking about use cases where an allocation can fail, instead of just terminating. So, implementation magic that turns exceptions into straight termination is not suitable. And we don't want to suggest that users attempt similar magic, because we don't actually want code to be conditionally-compiled depending on whether exceptions are enabled or not; we want something straightforward and portable, and we can easily have it: just add a

```
T* allocator::allocate_nothrow(size_t);
```

that returns `nullptr` if allocation fails. Also add a similar function to `allocator_traits`.

Make `allocator_traits` wrap allocators that don't opt in to that functionality, so that callers who absolutely don't want allocation exceptions can avoid them.

This part is worth closer consideration; the idea is that if exceptions as such are allowed, but not desirable getting out of an allocation operation such as the previously introduced `allocate_nothrow`, the question is, which allocators does that work with?

One option would be to make `allocate_nothrow` (when used via `allocator_traits`) always call `allocate_nothrow`. An alternative is to make `allocator_traits` call `allocate_nothrow` if it's callable, and otherwise

1. call the traditional `allocate`
2. and catch any exceptions and return a `nullptr` if an exception was thrown.

I'm not entirely sure which option is better. It seems reasonable to think that in systems where exceptions are not tolerated at all, there needs to be system-level knowledge that a non-throwing allocator is used. Therefore, presumably, on those systems the wrapping would never wrap a throwing allocator. However, systems that tolerate exceptions can use the suggested wrapping to be more compatible with existing allocators that haven't opted into non-throwing allocation as an addition. Users could, of course, provide custom specializations of `allocator_traits`.

Add a non-throwing reserve operation to vector and string.

This should be fairly straightforward; by now, we have established necessary allocator support, so we just expose it in facilities that can pre-allocate buffers. It's just a matter of adding a

```
bool reserve_nothrow(size_type);
```

Add a non-failing `push_back` to vector and string. This function will be UB to call if capacity isn't sufficient.

At this point, we'll take a little side-step; side-step in the sense that this suggestion has nothing to do with whether exceptions are tolerated or not, or desirable or not. The suggestion is that we allow users to avoid all error-checking overhead when they have made sure there is enough space for `N` `push_back` operations. So what we are looking at is

```
void push_back_unchecked(const T&);
```

So what we're looking at here is an unsafe zero-overhead `push_back`.

Add a `push_back` that can fail to vector and string. This function will report an allocation error, but will do so without an exception.

This suggestion is different from the previous one in the sense that the function would actually check for an error and report it. That is,

```
bool? push_back_nothrow(const T&);
```

Whether we want to add such functions depends on the next suggestion, but if we decide to add such functions, what should it

return? Should it wrap all exceptions, not just ones from an allocation, and wrap those into an 'expected'? Should it throw other errors but wrap allocation errors? I would like to point out that the last option is not as daft as it first seems; in general, users who don't want exceptions can, at least to some extent, use non-throwing types. Whether there are users who want to use throwing types but want non-throwing allocations, I don't know. Whether they want to combine all errors into non-throwing wrappings, I don't know. Whether a mixture of error reporting strategies is better, I don't know.

Add a `push_back` that can fail to vector and string. This function will have no effects if allocation fails, and will not throw.

Here we are looking at the following:

```
void push_back_nofail(const T&);
```

"Huh?", I hear you say. Well, the way to use such functions is

- get the size of the container
- do N `push_back_nofails`
- check whether the size of the container grew as expected
- if it didn't, some of the `push_back_nofails`, well, failed

The obvious downside is that we have lost all error information. But it's an alternative worth mentioning, because it serves certain kinds of repetitive bulk-ish operations decently well. It's not zero-overhead, either; failure in a `push_back_nofail` is not UB; it will merely not modify the container if it can't.

Decide what to do with non-contiguous containers, like lists and maps.

In non-contiguous containers, I think we don't have as many API choices as with vector and string. There is no reserve or any other preallocation, so any `insert_unchecked` that extends the container is a non-starter, as far as I can see. That leaves an insert that fails via a non-exceptional error, or an insert that doesn't report an error but doesn't modify the container either. Or, well, both, but this is where LEWG guidance is needed before going much further.

Feedback items requested

Before considering the list of feedback items, please take a look at the paragraph immediately below the list.

- Is `allocate_nothrow` an acceptable extension to `std::allocator`?
- Is `allocate_nothrow` an acceptable extension to `std::allocator_traits`, so that it transforms exceptions into `nullptr`?
- If not, should it by-default not do the transformation (but provide `allocate_nothrow` as a customization point), and would it then be acceptable?
- Should we have `push_back_unchecked` for vectors and strings?
- Should we have `push_back_nothrow` for vectors and strings?
- Should we have `push_back_nofail` for vectors and strings?
- Should we have `insert_nothrow` for non-contiguous containers?
- Should we have `insert_nofail` for non-contiguous containers?

Please note that the feedback requested doesn't necessarily mean that there's just a `push_back` and an `insert`. This is asking for API direction, and the exact functions to provide are TBD. The idea is to do small additions, not provide an alternative API for every operation, but the set of operations to provide is not cast in stone here. I want to first establish what kinds of APIs to provide, and then figure out what functions we really really should have.